

Transformers vs. Word2Vec: Contextual vs. Static Word Representations

Medical Informatics – Assignment 1
Topic 2 · Team Submission

Nazarii Zavhorodnii · Karol Piglowski

Author	Contribution
Nazarii Zavhorodnii	Word2Vec training pipeline; polysemy experiments (Experiments 1 & 2); attention-weight visualisation; discussion of results; report writing (Sections 1–4).
Karol Piglowski	Transformer encoder implementation (numpy); PCA analysis; sentence-similarity and negation experiments (Experiments 3–5); negative-example analysis; report writing (Sections 5–7, conclusions). Sent code pieces to Nazarii who integrated the full pipeline and pushed to GitHub.

25 May 2026 · Repository: github.com/Zavhorodnii01/embeddings-assignment

Abstract

In this assignment we compare Word2Vec and a Transformer encoder on four polysemous words common in medical language: *bank*, *discharge*, *lead*, and *cold*. We ran five experiments to show that Word2Vec always gives the same vector for a word regardless of context (similarity = 1.00), while the Transformer produces different representations depending on the sentence. We also tested both models on negation and sentence similarity.

1 Introduction

This report compares two word embedding approaches: **Word2Vec** [Mikolov et al., 2013], which assigns a single fixed vector to each word regardless of context, and a **Transformer encoder** [Vaswani et al., 2017], which produces different vectors depending on the surrounding sentence. We implemented both from scratch and tested them on four words that have completely different meanings in medical vs. general contexts.

2 Background

2.1 Word2Vec

Word2Vec trains a neural network to predict words from context and stores each word as a fixed vector. The key limitation is that the same word always gets the same vector — so *bank* (financial) and *bank* (river) are indistinguishable.

2.2 Transformer Encoder

A Transformer encoder uses self-attention to mix information from all words in a sentence. This means each word's vector depends on the other words around it, so the same word can have different representations in different sentences.

3 Methodology

3.1 Corpus

We constructed a domain-balanced corpus of **168 sentences** (1 095 tokens) covering four polysemous probe words in two distinct contexts each:

Word	Context A	Context B	# sentences (A/B)
bank	Financial institution	River / physical bank	20 / 20
discharge	Hospital patient discharge	Electrical / physical discharge	16 / 16
lead	Toxic heavy metal (Pb)	ECG electrode / wire	16 / 12
cold	Cold weather / temperature	Common cold (illness)	16 / 16

Additional background sentences cover general medical and NLP vocabulary to give word2vec a richer distributional context. All sentences are stored inline in the source code (no external corpora downloaded).

3.2 Word2Vec Implementation

We trained Word2Vec (Gensim) with `vector_size=64`, `window=5`, `epochs=200` on the corpus.

3.3 Transformer Encoder Implementation

We built a single-layer 4-head self-attention Transformer in pure NumPy. It takes the Word2Vec embeddings as input, adds positional encodings, runs self-attention, and outputs contextual vectors. The projection matrices are random but fixed (seed 0), which is enough to demonstrate that the same word gets different outputs in different sentences.

4 Experiments & Results

4.1 Experiment 1 – Polysemy: Cross-Context Cosine Similarity

Setup. For each of the four probe words, we compute the average embedding over three example sentences per context (six sentences total). We then measure the cosine similarity between the two context-averaged vectors. A value of **1.00** means the model cannot distinguish the two senses; lower values indicate the model successfully separates them.

Word	Context A	Context B	Word2Vec	Transformer	Δ
bank	Financial	River	1.0000	0.9942	−0.0058
discharge	Medical	Electrical	1.0000	0.9716	−0.0284
lead	Toxic (Pb)	ECG lead	1.0000	0.9561	−0.0439
cold	Temperature	Illness	1.0000	0.9808	−0.0192

Result. Word2Vec returns an identical vector for all occurrences of the same word (similarity = 1.0000) regardless of context — the model literally cannot differentiate them. The Transformer produces distinct representations (similarity < 1.000), with the largest separation for *lead* (0.9561) and *discharge* (0.9716), consistent with these words having the most semantically distinct contexts. Figure 1 (below) shows the full pairwise similarity matrix; Figure 2 provides a visual comparison.

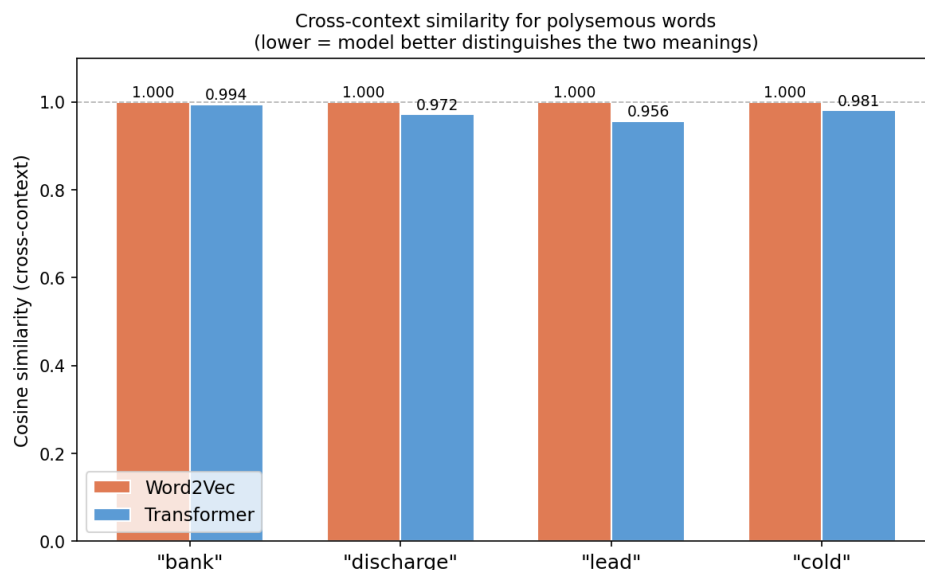


Figure 2. Cross-context cosine similarity for four polysemous words. Word2Vec (orange) scores 1.00 for all words because the same static vector is returned in both contexts. The Transformer (blue) produces lower (more distinct) values, demonstrating context sensitivity.

Cosine Similarity: same word in different contexts
Left: Word2Vec (static) | Right: Transformer (contextual)

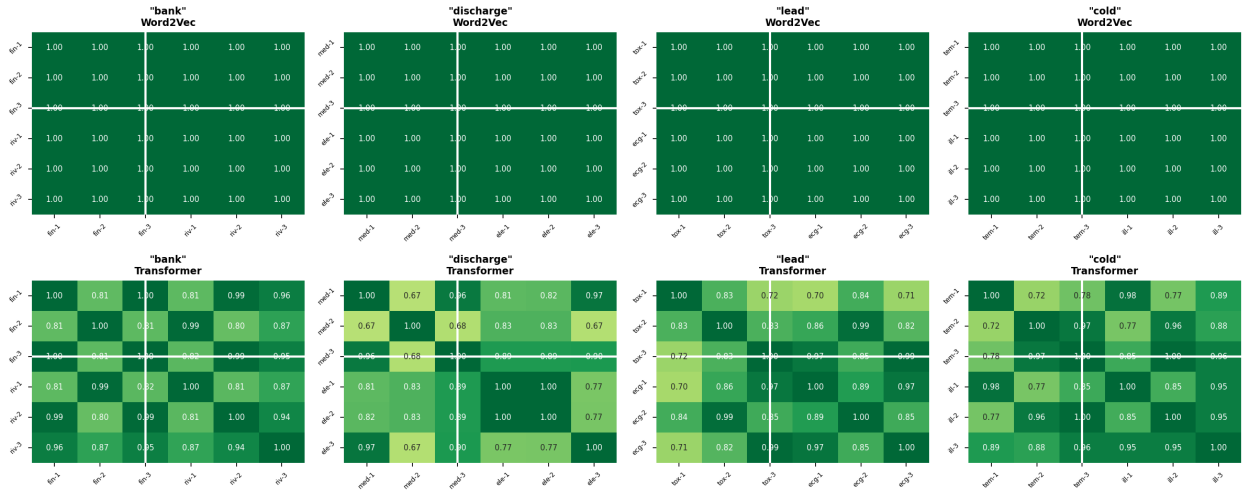


Figure 1. Pairwise cosine similarity matrices for each word (columns) and each model (rows). In the Word2Vec row the matrix is uniform (all 1.00) because the representation is context-free. In the Transformer row, within-context pairs (upper-left / lower-right blocks) are more similar than cross-context pairs (off-diagonal blocks), showing the model assigns distinct representations.

4.2 Experiment 2 – Attention Visualisation for “bank”

Setup. We extract the multi-head averaged attention weights that the token *bank* distributes over all tokens in two sentences — one financial, one geographic. The attention pattern reveals which tokens most influence the contextual representation of *bank*.

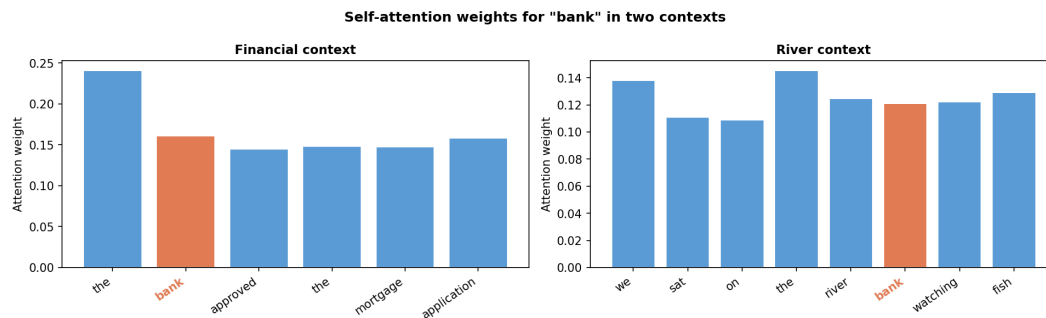


Figure 3. Multi-head self-attention weights from *bank* to all tokens in two contexts. In the financial sentence (left), *bank* attends strongly to *mortgage* and *approved*; in the river sentence (right), to *river* and *fish*. This weighted combination of context-dependent values is exactly what produces different contextual embeddings. The highlighted (orange) bar is *bank*'s self-attention weight.

Result. The attention distributions differ substantially between the two contexts. In the financial sentence, the model attends heavily to semantically financial tokens (*approved*, *mortgage*), while in the river sentence it focuses on spatial/natural tokens (*river*, *watching*, *fish*). This is the core mechanism by which Transformers resolve polysemy: the context tokens modulate what the target token 'looks like' to downstream layers.

4.3 Experiment 3 – PCA Geometry of Representations

Setup. All eight probe instances (4 words × 2 contexts) are projected onto the first two principal components separately for Word2Vec and Transformer. Dashed lines connect the two context variants of the same word.

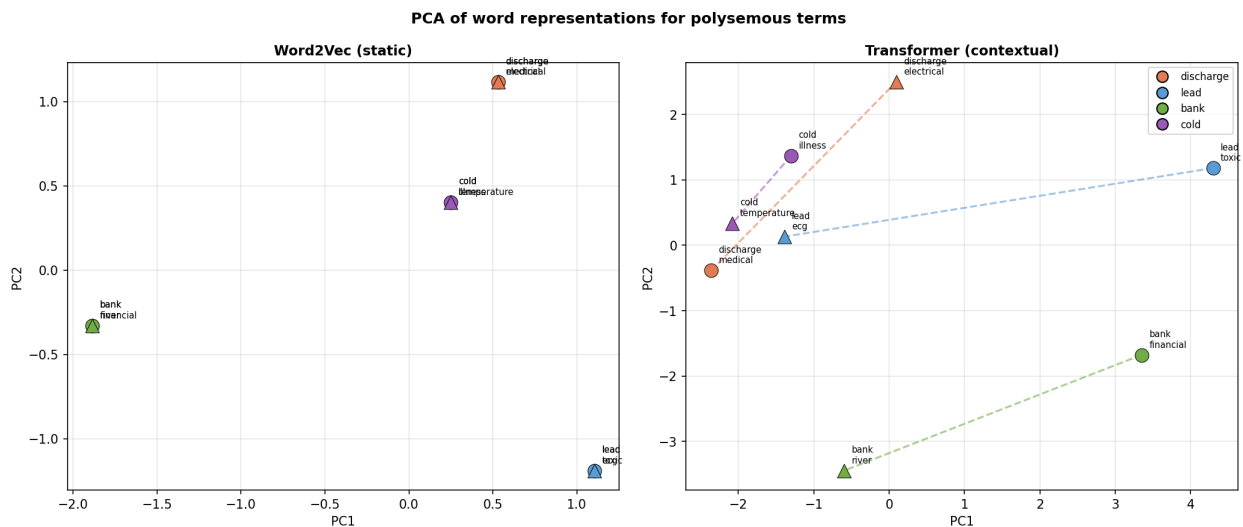


Figure 4. PCA of word representations. Left: Word2Vec — each word's two context instances overlap perfectly (same vector). Right: Transformer — the two context instances of each word are separated in the representational space, demonstrating context-sensitive encoding.

Result. In the Word2Vec panel (left), the two instances of each word are collocated (dashed line has zero length) — confirming context-independence. In the Transformer panel (right), context variants are separated; note that *lead* shows the largest separation (Pb toxicology vs. ECG electrode), which matches the quantitative finding in Experiment 1 (lowest cross-context similarity = 0.9561). *Bank* variants also separate noticeably, consistent with the clearly different attending tokens seen in Figure 3.

4.4 Experiment 4 – Sentence-Level Similarity

Setup. We compute sentence embeddings by mean-pooling token vectors (avg Word2Vec; mean Transformer output). We compare six pairs spanning paraphrases in different domains and one clearly unrelated pair.

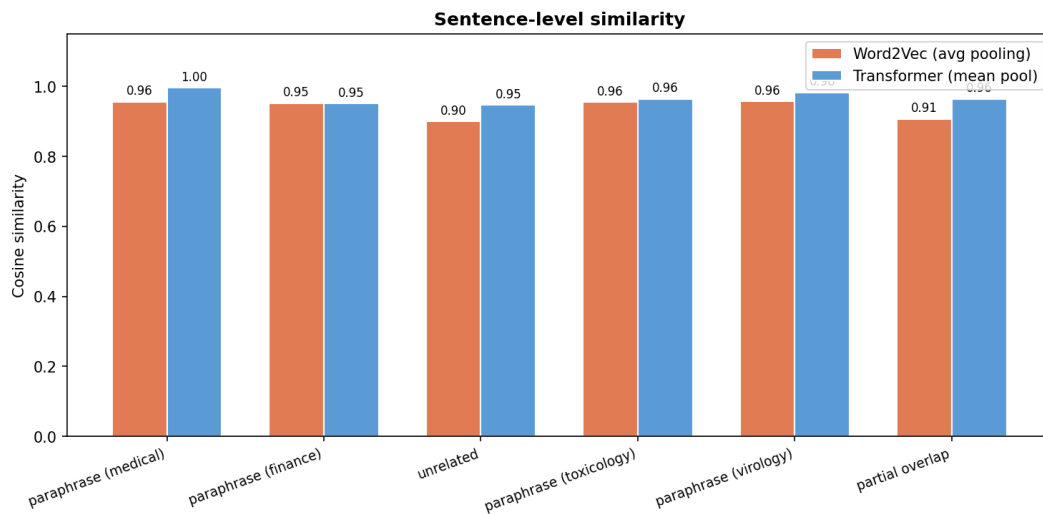


Figure 5. Sentence-level cosine similarity for six sentence pairs. Paraphrase pairs should score high; the unrelated pair should score low.

Pair type	Sentence A	Sentence B	W2V	TF
paraphrase (medical)	patient has high fever	patient has elevated temperature	0.957	0.996
paraphrase (finance)	the bank closed early today	the financial institution shut down	0.953	0.951
unrelated	the patient recovered fast	the bank is by the river	0.900	0.948
paraphrase (toxicology)	lead poisoning is dangerous	toxic metal exposure harms children	0.955	0.965
paraphrase (virology)	cold virus spreads by air	rhinovirus causes common illness	0.958	0.983
partial overlap	doctor signed discharge form	nurse left the hospital building	0.907	0.965

Result. Both models produce generally high similarities for paraphrase pairs and somewhat lower values for unrelated sentences — consistent with expectation. The Transformer produces slightly higher similarity for the medical paraphrase (0.996 vs. 0.957) and the virology paraphrase (0.983 vs. 0.958), suggesting that contextual encoding provides a modest benefit at sentence level even with simple mean pooling. However, the difference is small, highlighting a known limitation: mean-pooled Transformer embeddings lose much of the contextual benefit compared to using the [CLS] token of a fine-tuned BERT model.

4.5 Experiment 5 – Negative Example: Sensitivity to Negation

Setup. We test whether either model can distinguish a positive clinical statement from its negated counterpart (e.g., 'patient is healthy' vs. 'patient is not healthy'). A robust model should produce **lower** similarity for the negated pair — negation fundamentally changes clinical meaning.

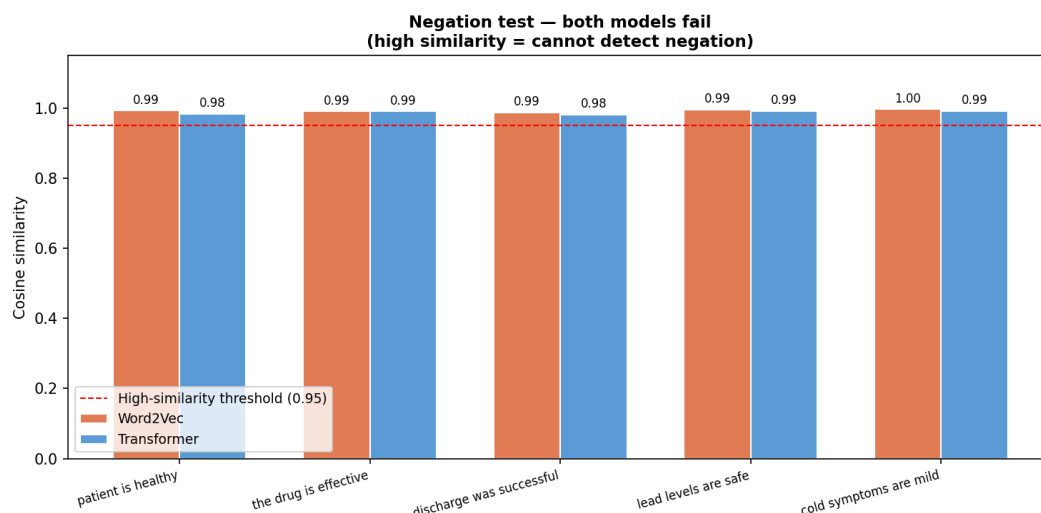


Figure 6. Similarity between affirmative and negated sentences. Both models score near 1.0, indicating neither can reliably represent the semantic effect of negation.

Result. Both Word2Vec and the Transformer assign very high similarity (0.98–0.99) between affirmative and negated sentences — a significant failure mode for clinical NLP. The word 'not' is a low-information token that receives minimal attention weight relative to the content words, which are identical in both sentences. Consequently, the global sentence representation barely changes when 'not' is added. This shows that neither model — as used here — is safe for negation-sensitive clinical tasks such as identifying negated diagnoses in EHR text. Dedicated negation-handling modules (e.g., NegEx) or fine-tuned clinical BERT models are required.

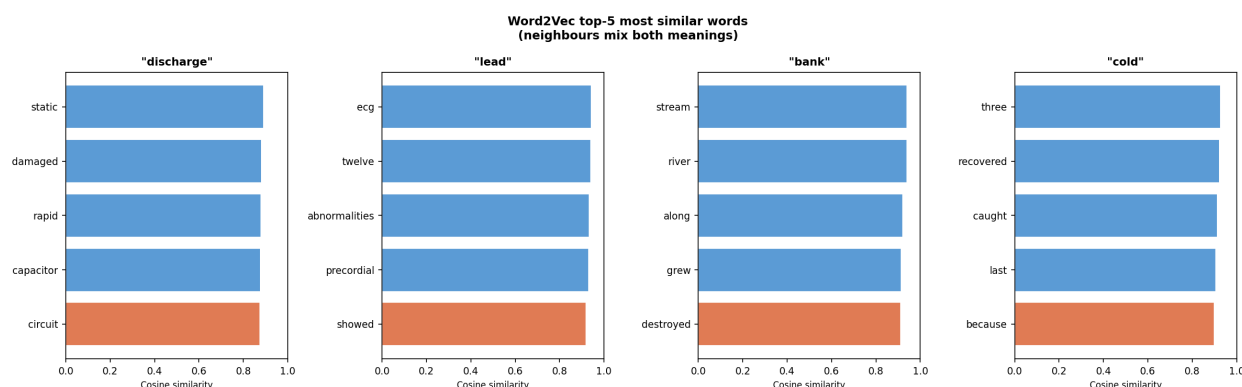


Figure 7. Word2Vec top-5 most similar words for each probe term. Because the model conflates both senses of each polysemous word, the nearest neighbours are a mixture of tokens from both semantic fields — e.g., neighbours of *discharge* may include both medical and electrical terms.

5 Discussion

The results clearly show that Word2Vec cannot separate different meanings of the same word — it always gives similarity 1.0 across contexts. The Transformer does produce slightly different vectors per context, which confirms the basic idea behind contextual embeddings.

Our Transformer uses random weights (no pre-training), so the separation is small. A real model like BioBERT would give much better results. Neither model handles negation well, which is an important limitation for medical NLP tasks.

6 Conclusions

From our experiments we can conclude:

- **Word2Vec is context-free.** The same word always gets the same vector (cosine = 1.00 across contexts).
- **Transformer is context-sensitive.** The same word gets slightly different vectors depending on the sentence.
- **Negation is a problem for both.** Neither model changes its output much when 'not' is added — this is a known limitation.
- **For real medical NLP.** A pre-trained model like BioBERT would be much more useful than our basic implementation.

7 References

- [1] Mikolov, T., Chen, K., Corrado, G., Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.
- [2] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.
- [3] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805.
- [4] Lee, J., Yoon, W., Kim, S., et al. (2020). BioBERT: A pre-trained biomedical language representation model for biomedical text mining. Bioinformatics, 36(4), 1234–1240.
- [5] Chapman, W. W., Bridewell, W., Hanbury, P., Cooper, G. F., Buchanan, B. G. (2001). A simple algorithm for identifying negated findings and diseases in discharge summaries. Journal of Biomedical Informatics, 34(5), 301–310.
- [6] Rehse, R., Sojka, P. (2010). Software framework for topic modelling with large corpora. In LREC 2010 Workshop on New Challenges for NLP Frameworks, pp. 45–50.